ENGR 102 Summer Bridge
Day 12 Student Workbook

For Loops, Range, Counters, and Accumulators

Tuesday, July 21, 2026 • 50 points

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Daily target — increasing independence

Build fixed-count programs that preserve loop state and exact bounds.

Allowed tools	Day 10 tools plus for, range, counters, and accumulators
Support supplied	Required inputs, required outputs, and one sample run are supplied.
You must decide	Initialization, range bounds, accumulator updates, and the placement of the threshold decision.
Scaffold level	State inventory prompted once; loop body is student-authored.

Scoring and completion standard — 50 points

Evidence	Points	Secure work shows
Integrated trace	10	intermediate state, condition results, and exact output
Diagnosis and repair	10	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	30	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace — 10 points

Trace the range, the conditional counter, and the accumulator after every pass.

```
total = 0
multiple_count = 0

for number in range(2, 11, 2):
    if number % 4 == 0:
        multiple_count = multiple_count + 1
    total = total + number - multiple_count
    print(number, total, multiple_count)
```

Your task

1. List the values produced by range(2, 11, 2).
2. Create one pass ledger containing number, multiple_count after the if, and total after the update.
3. Write all printed lines in exact order.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair — 10 points

The intended program must add the integers from 1 through limit, inclusive.

```
limit = int(input())
total = 0

for number in range(1, limit):
    total = total + number

print(total)
```

Your task

1. For limit = 4, state the actual and intended totals and identify the exact cause.
2. Write the minimal repair and a boundary test that would expose the original bug.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program — 30 points across Pages 4-5

Program specification

- Read sample_count as an integer and threshold as a float.
- If sample_count is not positive, print Invalid count and do not ask for readings.
- Otherwise read exactly sample_count floating-point readings.
- Compute the total, average, and number of readings at or above threshold.
- Print total, average, and at_or_above on separate lines in that order.

Required planning evidence

1. Name the state that must exist before the loop and give each initial value.

Program workspace – begin the connected solution here.

Task 3 continued. Test and defend the program **included in 30 points**

Continue code if needed.

Required tests, expected results, and brief defense